PROJECT WORK CS6811

THIRD REVIEW

Federated Learning for Code Generation with Hallucination-Aware Fine Tuning

TEAM DETAILS

Abhinavh P (2022503059),

Prabhakara Arjun R (2022503003),

Buvanes Srivardhan K (20225033037).

MENTOR

Dr. B. Thanasekhar,

Professor,

Computer Technology, MIT

PROBLEM STATEMENT

Large Language Models (LLMs) used for code generation often produce hallucinated outputs, resulting in syntactically correct but semantically incorrect code. These inaccuracies reduce the reliability of such models in real-world software development scenarios.While existing approaches attempt to improve performance through fine-tuning and prompt engineering, they do not effectively leverage distributed data available across multiple clients.

In practical environments, data required for improving model performance is decentralized and cannot be shared due to privacy and ownership constraints. Although federated learning provides a solution for decentralized training, traditional aggregation methods such as Federated Averaging (FedAvg) rely only on dataset size and fail to consider variations in error patterns across clients. This limits the model's ability to learn effectively from diverse hallucination distributions.

Therefore, there is a need for a privacy-preserving framework that not only enables collaborative learning across distributed clients but also incorporates error-aware aggregation strategies to improve the reliability and correctness of LLM-based code generation.

OBJECTIVES

The objective of this project is to develop a federated learning-based framework to improve the reliability of large language model (LLM)-based code generation, with a primary focus on parameter-efficient fine-tuning using Low-Rank Adaptation (LoRA). The framework aims to enable collaborative model training across multiple decentralized clients, allowing local fine-tuning on private codebases while preserving data privacy and reducing computational and communication overhead.

Another objective is to design and integrate a hallucination-aware pipeline that detects and classifies errors in generated code using both static and dynamic program analysis techniques. This includes identifying syntactic, semantic, API-related, and logical hallucinations, and establishing a structured taxonomy to support consistent evaluation and targeted model refinement.

The project further aims to develop an error-aware aggregation strategy, termed Fed-Error Average, which incorporates the distribution of hallucination errors from different clients during federated aggregation. This is intended to improve learning from heterogeneous and non-IID data distributions and enhance the robustness of the global model.

LITERATURE SURVEY

S.NO	TITLE	NAME OF THE JOURNAL	METHODOLOGY	LIMITATIONS
[1]	Advancing LLM-Generated Code Reliability: A Hybrid Approach for Hallucination Detection	IEEE Transactions on Software Engineering (TSE), 2025	• Proposes SDHD, a hybrid hallucination detection framework that integrates static analysis and dynamic execution for LLM-generated code. . • This framework detects eight hallucination types and achieves superior precision, recall, and F1-score across multiple benchmarks.	• Dynamic detection depends on test coverage quality, and incomplete coverage may miss certain logical failures. . • The framework is currently validated only on Python, limiting cross-language generalizability.
[2]	LLM-Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation	IEEE Transactions on Software Engineering (TSE), 2024	• TICODER is a test-driven interactive workflow that incrementally clarifies user intent through LLM-generated tests and lightweight user feedback. • The system ranks and prunes candidate code suggestions based on user-approved tests using two variants: PASS/FAIL validation and explicit output specification.	• There is interactive overhead, and incorrect or noisy user feedback can prune correct solutions prematurely. • Effectiveness is contingent on the LLM's ability to generate high-quality tests.
[3]	A Federated Adaptive Large Language	IEEE Transactions on Services	• The framework integrates LoRA-based parameter-efficient fine-tuning within a federated setting, where	• Experimental results show degraded

	Model Fine-Tuning Framework for Software Development	Computing, 2025	each organization locally trains lightweight adapter modules and shares only adapter to the central server. • A FedAvg-style is used to combine adapters based on local data size and an L2-norm constraint to manage gradient divergence	performance on unrelated datasets. • Although communication costs are reduced, the framework still requires multiple communication rounds, which may introduce latency in large-scale federations.
[4]	Towards Efficient Fine-Tuning of Language Models With Organizational Data for Automated Software Review	IEEE Transactions on Software Engineering 2024	• Proposes CodeMentor, a framework for few-shot learning that utilizes heuristic rules and weak supervision to construct datasets from internal organizational data • It employs Low-Rank Adaptation (LoRA) for parameter-efficient fine-tuning and integrates Reinforcement Learning from Human Feedback (RLHF) to incorporate domain expertise.	• The framework depends on the quality of organizational data, which may introduce domain bias and limit generalization across different coding lang. • RLHF requires computational resources and expert involvement,bottle neck scenario during scaling.
[5]	The Impact of Prompt Programming on Function-Level Code Generatio	IEEE Transactions on Software Engineering (TSE), 2025	• Empirical study to evaluate the impact of prompt programming techniques on function-level code generation. • CodePromptEval, a dataset of 7,072 prompts derived from 221 real-world Python functions, covering five prompt techniques: few-shot learning, chain-of-thought, persona, function signature, and package context, including all 32 possible combinations.	• The study is restricted to Python function-level tasks, limiting generalizability to other languages and higher-level program synthesis. • Rely on benchmark-driven evaluation, which may

				not fully capture real-world scenario. • Prompt techniques are evaluated in a fixed ordering, excluding effects of alternative prompt sequencing.
[6]	An Adaptive Framework Embedded With LLM for Knowledge Graph Construction	IEEE Transactions on Multimedia, Vol. 27, 2025	• KG construction happens in three stages: Information Extraction (IE) for open triple extraction, Additional Relation (AR) for enriching triples with semantic information, and Knowledge Graph Normalization (KGN) for schema-level alignment using vector similarity search. • Adaptive Prompt Generation (APG) module automatically generates, evaluates, and ranks prompts to guide LLM behavior across stages, enabling multi-domain KG construction from text, speech, and images.	• High computational overhead due to multi-stage LLM inference, embedding generation, and vector similarity matching. • The framework relies heavily on prompt engineering, which,, is not fully autonomous and may still suffer from hallucination and semantic inconsistency.

[7]	Fight Fire With Fire: How Much Can We Trust ChatGPT on Source Code-Related Tasks?	IEEE Transactions on Software Engineering (TSE), Vol. 50, No. 12, 2024	• Evaluating self-verification capability of GPT for: code generation, code completion, and program repair. • Direct question, guiding question, and test report generation prompting is used assess whether ChatGPT can reliably validate its own outputs. • Effectiveness is measured using accuracy, precision, recall, and F1-score, with additional analysis of self-contradictory hallucinations.	• Results show that ChatGPT frequently overestimates the correctness of its own outputs, leading to low recall in direct self-verification. • While guiding questions and test reports improve recall, precision is reduced. • Generated test reports often contain incorrect explanations, especially for code generation and failed repairs. .
[8]	FedALoRA: Adaptive Local LoRA Aggregation for Personalized Federated Learning in LLM	IEEE Internet of Things Journal, Vol. 12, No. 24, December 2025	• Proposes FedALoRA, a personalized federated learning framework that integrates LoRA-based PEFT with adaptive local aggregation to address severe non-IID data in federated LLM training. • FedALoRA uses an Adaptive LoRA Aggregation to merge global and local parameters via locally learned weights. This selective, element-wise	• FedALoRA introduces additional local optimization complexity due to adaptive weight learning in ALoRA modules.

			fusion focused on higher transformer layers preserves domain expertise while capturing shared knowledge without increasing communication costs.	• The method requires careful tuning of the hyperparamet er *l*, which controls the number of transformer layers involved in adaptive aggregation, and improper settings can degrade performance.
[9]	No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT	IEEE Transactions on Software Engineering (TSE), Vol. 50, No. 6, June 2024	• Evaluation framework for assessing ChatGPT-based code generation. The authors construct standardized prompts for 728 LeetCode algorithm problems and 54 CWE security scenarios across five programming languages. • Generated code is evaluated using automated judgment platforms (LeetCode for functional correctness) and static analysis tools (CodeQL for vulnerability detection). • A multi-round fixing process is designed, where execution or analysis feedback is iteratively fed back	• ChatGPT performs significantly better on problems published before 2021, indicating reliance on memorized patterns rather than reasoning. • The dialog-based fixing process has limited effectiveness, for logically complex problems . • Generated code often contains security vulnerabilities and non-deterministic behavior.

[10]	Knowledge Enhanced Program Repair for Data Science Code	Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE), 2025	• API KG Construction is done using a custom ontology and RDF triples to model API attributes and dependencies for seven data science libraries. • SPARQL is used for knowledge retrieval from on buggy code. • AST level Bug Knowledge Enrichment, which executes code node-by-node to localize runtime and assertion errors at the AST node level.	• DS-KG is version-specific and requires continuous updates to track evolving API documentation, making long-term maintenance costly and error-prone. • KG-based knowledge retrieval introduces higher latency than plain-text search
[11]	THINK: Tackling API Hallucinations in LLMs via Injecting Knowledge	IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2025	• THINK proposes a two-phase API hallucination mitigation framework consisting of Pre and Post execution Optimization. • Constructs an API knowledge database (Javadoc + validated code examples) and retrieves task-relevant APIs using task decomposition, similarity search, and LLM-based API labeling to guide code generation. • Error Classification into Seven API Error Types for correction	• It may introduce irrelevant API knowledge, occasionally degrading performance for tasks already solvable by strong LLMs. • Both retrieval results and ReAct-based repair depend on LLM generation, leading to non-deterministic behavior and reduced reproducibility across runs

[12]	EvoAPR: Enhancing Large Language Models for Automatic Program Repair with Genetic Algorithm and Dynamic LoRA	IEEE International Conference on Web Services (ICWS), 2025	• Genetic evolution of buggy hunks under tri-objective fitness: syntax compliance, bug-hunk masking rate, repair-location masking rate. • Dynamic LoRA swaps non-overlapping rank-r adapters per bug-index/project for on-demand, project-specific parameter retrieval. • Base + LoRA models generate candidate patches; token-level entropy scorer picks the least uncertain fix as the final patch.	• Genetic template evolution and dynamic LoRA fine-tuning introduce significant computation and implementation complexity compared to direct prompting-based APR methods. • The effectiveness of both stages relies on accurate bug localization and rich bug context.

[13]	CodeHalu: Investigating Code Hallucinations in LLMs via Execution Based Verification	Proceedings of the AAAI Conference on Artificial Intelligence (AAAI), 2025	• Executes LLM-generated code against test cases under time and memory constraints, recording execution failures, unexpected outputs, infinite loop behaviors to detect hallucination states. • Aggregates execution states across multiple LLMs and applies a two-stage heuristic induction process to classify hallucinations into four main types (Mapping, Naming, Resource, Logic) and eight subcategories.	• The study focuses exclusively on Python, and hallucination behaviors in other programming languages are not evaluated. • CodeHalu targets functional and logical correctness; identifying or mitigating security vulnerabilities is outside its scope.

ARCHITECTURE DIAGRAM OF THE PROPOSED SYSTEM

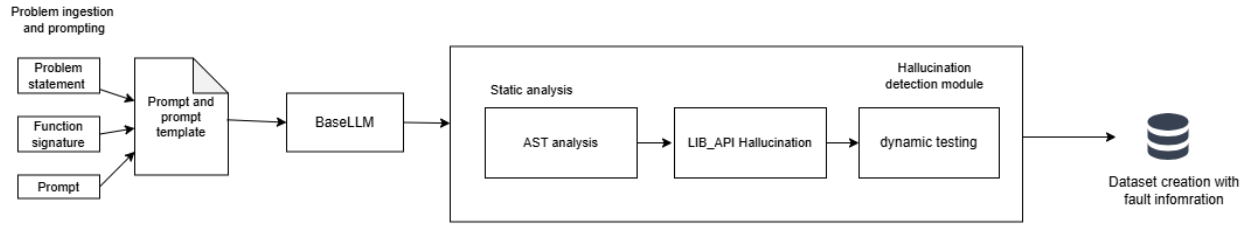

***Figure 1**: Client side hallucination detection with Dataset creation*

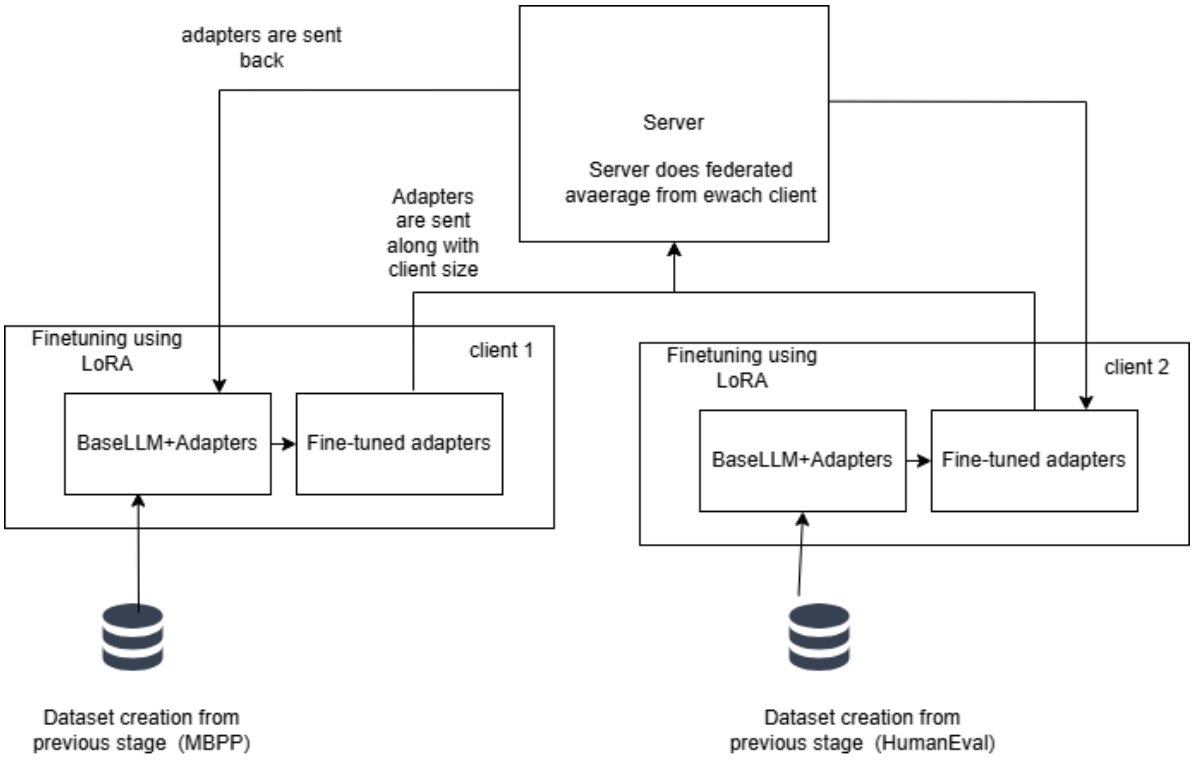

***Figure 2**: Federated learning + LoRA for fine-tuning*

Tools Used:

Google Colab

Google Colab is a cloud-based Jupyter Notebook environment that allows users to write and execute Python code interactively. It provides free access to GPU and TPU resources, making it suitable for training deep learning models. During the initial stages of this work, Google Colab was used with a T4 GPU for code generation and early experimentation, enabling rapid prototyping without requiring local hardware setup.

Visual Studio Code and git

Visual Studio Code (VS Code) is a lightweight but powerful source code editor developed by Microsoft. It was used as the primary development environment for hallucination detection and analysis, as well as for results interpretation. VS Code was used in conjunction with Git and GitHub for version control and file management, ensuring systematic tracking of changes across the codebase. The implementation of federated average and federated error averaging was also carried out within this environment.

JupyterHub

CTSKII JupyterHub is an institutionally hosted JupyterHub platform that provides users with isolated, containerized JupyterLab environments backed by persistent storage volumes. Each user is allocated a dedicated Docker container with a private home directory, ensuring data persistence across sessions. The platform offered access to an NVIDIA A4000 GPU at no cost, making it well suited for computationally intensive tasks. Local session storage was available upon server allocation, and user files were retained even after container restarts. This platform was used for fine-tuning experiments and federated averaging, leveraging the A4000 GPU for efficient model training and verification.

Dataset Used:

Dataset	**Source**	**Task Count**	**Domain**	**Used For**
Human Eval	*Hugging Face (openai/evals)*	164	Functional correctness	*Code generation, hallucination detection, SFT*
MBPP	*Hugging Face (google-research-datasets/mbpp, sanitized)*	377	Basic algorithms	*Code generation, hallucination detection, SFT*
DS-1000	*Hugging Face (xlangai/DS-1000)*	1000	Data science (NumPy, Pandas, etc.)	*Code generation, hallucination detection, SFT*

Table 1: Datasets Used and their purpose

1. **HumanEval**

- task_id: Unique identifier (e.g., HumanEval/0)
- prompt: Natural language task description with docstrings/examples used for code generation
- canonical_solution: Reference Python implementation for training and evaluation
- test: Unit test code with assertions for dynamic execution
- entry_point: Function name under test (e.g., has_close_elements)

2. **MBPP**

- source_file: Origin file or notebook of the task
- task_id: Unique task ID (e.g., 602, 603)
- prompt: Short natural language description used for code generation
- code: Reference implementation used during fine-tuning
- test_imports: Required import statements for running tests
- test_list: List of assert statements for validation

3. **DS-1000**

- prompt: Problem description with instruction (e.g., assign result to a variable)
- reference_code: Ground-truth implementation for evaluation/training
- metadata: Library/category info (NumPy, Pandas, etc.)
- code_context: Execution setup including helpers and input generation
- generated_code_snippet: Partial generated solution (if applicable)
- full_code: Complete generated solution used for evaluation
- task_id: Unique ID (e.g., DS0000, DS0132)
- Status: Execution result (e.g., passed, failed)

Model Used:

Qwen2.5-Coder is the latest series of Code-Specific Qwen large language models (formerly known as CodeQwen). Significantly improvements in **code generation**, **code reasoning** and **code fixing**. Base on the strong Qwen2.5, we scale up the training tokens into 5.5 trillion including source code, text-code grounding, Synthetic data, etc. Qwen2.5-Coder-32B has become the current state-of-the-art open-source codeLLM, with its coding abilities matching those of GPT-4o.

Attribute	Specification
Model Name	Qwen2.5-Coder-3B-Instruct
Parameters	~3 billion
Architecture	Transformer (36 layers)
Framework	Hugging Face Transformers
Fine-tuning	LoRA / QLoRA (4-bit quantization)
LoRA Rank	16
LoRA Alpha	32
Projections	Q, K, V, O, up, down, gate (7 per layer)

MODULE WISE IMPLEMENTATION

Module 1: Code Generation Module

The code generation module loads the three benchmark datasets and prompts the frozen Qwen2.5-Coder-3B-Instruct model to produce initial code solutions. The model operates in deterministic mode using greedy decoding (*do_sample=False, temperature=1.0*) to ensure reproducibility. Each dataset is loaded from the Hugging Face datasets library and handled according to its own prompt structure before being passed to the model.

For HumanEval, the dataset is loaded from openai/openai_humaneval. The prompt column, containing the natural language description and docstring, is passed directly to the model. The entry_point column, identifying the function under test, is retained alongside the generated code for use by the dynamic execution module during hallucination detection.

For MBPP, the sanitized split is loaded from google-research-datasets/mbpp. A preprocessing step extracts the function signature from the reference code by scanning for the first line beginning with def, which is then appended to the prompt. This ensures the generated code conforms to the function name and argument structure expected by the benchmark test cases.

For DS-1000, loaded from xlangai/DS-1000, the verbose prompt is preprocessed by splitting on "A:" and retaining only the question portion as prompt_v2. The code_context column is preserved for dynamic testing, and each task is assigned a unique identifier in the format DS0000–DS0999 for consistent tracking.

Module 2: Hallucination Detection Module

The AST module is the first stage of hallucination detection and inspects the generated code statically without executing it. If parsing fails due to a syntax or indentation error, the error type, message, and line number are recorded and all remaining stages are skipped. When parsing succeeds, the tree is traversed to check that control-flow constructs are used correctly — return only inside functions, break and continue only inside loops — and flow-aware name analysis is performed across all control paths to detect variables used without being defined. If any errors are found, fault information is built immediately and the pipeline exits early.

```
AST_ANALYSIS_OUTPUT_SCHEMA = {
    "ast_parsed": bool,
    "ast_errors": list[{
        "type": str,
        "start_line": int,
        "end_line": int,
        "col_offset": int | None,
        "message": str
    }]
}
```

***Figure 3**: AST Analysis Output Schema*

If the AST stage passes, the dynamic module executes the generated code against the benchmark's official test cases in an isolated environment with a ten-second timeout. Execution is dispatched by dataset type: HumanEval uses the test column and entry_point to invoke the function under test; MBPP uses assert statements from test_list along with imports from test_imports; DS-1000 inserts the generated solution into the code_context harness before execution. If all tests pass, the task is marked as passed. If any test fails or raises an exception, the status, exception type, error message, line number, and for assertion failures the expected and actual outputs are all recorded.

```
DYNAMIC_ANALYSIS_OUTPUT_SCHEMA = {
    "status": str,              # "passed" | "failed"
    "error_type": str,          # RuntimeError | AssertionError |
                                # SyntaxError | Timeout |
                                # TestParseError | UnknownDataset |
                                # None (if passed)

    "error_message": str,
    "line_number": str | int,
    "test_case": str,
    "testcase_output": str,
    "generated_code": str,
    "dataset": str,
    "task_id": str
}
```

***Figure 4**: Dynamic Analysis Output Schema*

If the dynamic stage reports a failure, the LIB_API module is invoked to statically validate how external libraries are used. It checks that imported modules exist and can be loaded, that accessed attributes are valid on their objects, and that function and method call signatures match the expected argument counts and types. It uses importlib to load referenced modules and inspect to retrieve real signatures, ensuring only genuine API hallucinations are flagged rather than environment-level missing packages. The output records the error category, the object or function

involved, and the line number for each issue. Results from all three stages are then assembled into a unified fault record passed to the dataset creation module.

```
LIB_API_ANALYSIS_OUTPUT_SCHEMA = {
    "libapi_analyzed": bool,

    "name_error": int,
    "attribute_error": int,
    "module_not_found": int,

    "total_libapi_errors": int,

    "libapi_details": list[{
        "type": str,
        "name": str | None,
        "object": str | None,
        "attribute": str | None,
        "line": int
    }]
}
```

***Figure 5**: Library API Output Schema*

LoRA Fine-Tuning

Parameter	Value
r (rank)	16
lora_alpha	32
target_modules	7 projections (q_proj, v_proj, k_proj, o_proj, gate_proj, up_proj, down_proj)
lora_dropout	0.05
bias	"none"
task_type	CAUSAL_LM

***Table 3**: LoRA Parameters and Values used*

The rank *r* sets the inner size of the low-rank matrices A and B; smaller *r* reduces parameters, larger *r* increases expressiveness. *lora_alpha* scales the LoRA update (here alpha/r = 2) and controls how strongly the adapter changes the base model. The seven target modules (q_proj, v_proj, k_proj, o_proj, gate_proj, up_proj, down_proj) are the attention and feedforward layers where LoRA is applied. *lora_dropout* adds dropout in the LoRA branch to reduce overfitting. *bias* is set to "none" so bias terms stay fixed and the number of trainable parameters stays small. *task_type* is CAUSAL_LM so PEFT treats the task as causal next-token prediction instead of classification.

Parameter	Value
num_train_epochs	3
per_device_train_batch_size	2
gradient_accumulation_steps	4
learning_rate	2e-4
weight_decay	0.01
warmup_ratio	0.03
lr_scheduler_type	cosine
max_grad_norm	0.3
max_seq_length	2048
packing	False
bf16 / fp16	auto-detect
save_strategy	epoch
save_total_limit	1
logging_steps	5

***Table 4**: Training hyperparameters and optimization configuration used for LoRA-based model fine-tuning.*

num_train_epochs is 3, so the model sees the full training set three times to learn from the hallucination-aware data. per_device_train_batch_size is 2 to keep memory use within GPU limits (e.g. T4), and gradient_accumulation_steps is 4, so gradients are accumulated over four forward passes before one backward step, giving an effective batch size of 8. learning_rate is set to 2e-4 because only the small LoRA adapter is trained, and weight_decay is 0.01 to apply L2 regularization and reduce overfitting. warmup_ratio of 0.03 means the learning rate increases linearly from zero to its peak over the first 3% of training steps, and lr_scheduler_type is cosine so the rate decays smoothly from peak to near zero. max_grad_norm is 0.3 for gradient clipping, which limits large updates and helps stability during LoRA fine-tuning. max_seq_length of 2048 truncates longer examples and keeps computation manageable. packing is False so each prompt–solution pair is a separate training example, which fits the completion-only loss collator. bf16 or fp16 is chosen automatically depending on GPU support to reduce memory and allow larger batches. save_strategy is epoch to save checkpoints at the end of each epoch, and save_total_limit is 1 so only the latest checkpoint is kept. logging_steps is 5 so loss and other metrics are logged every 5 optimizer steps for monitoring.

Finally , the three datasets are fine-tuned independently, each producing its own set of LoRA adapters.

Module 3: Federated Aggregation Module

Three datasets are fine-tuned independently, each producing its own set of LoRA adapters, which are then aggregated using Federated Averaging (FedAvg) implemented in two distinct

approaches.To validate the system, an initial evaluation was conducted by training and testing on the same dataset, confirming overall improvement across the federated pipeline. However, this approach alone is insufficient to rule out overfitting — a model could simply be memorising training samples rather than learning generalisable patterns.

To address this, a 75/25 train-test split was introduced. Each dataset is fine-tuned exclusively on its training partition, and evaluation is performed on the held-out test partition. This setup allows a direct comparison of baseline model performance against post-aggregation performance on unseen data. An improvement in test-set metrics after FedAvg aggregation — data the model was never trained on — demonstrates that the fine-tuned adapters have learned transferable coding patterns rather than overfitting to the training samples.

```
Sample counts (n_k): {'mbpp': 327, 'humaneval': 164, 'ds1000': 1000}
Weights (alpha_k): {'mbpp': 0.2193158953722334, 'humaneval': 0.10999329309188464, 'ds1000': 0.670690811535882}
```

***Figure 6.1.a**: Output of fine-tuned adapters after Federated average (complete dataset)*

```
Final alpha_k: {'mbpp': 0.1765233238912576, 'humaneval': 0.039429434864408704, 'ds1000': 0.7840472412443337}
```

***Figure 6.1.b**: Output of fine-tuned adapters after Federated Error average (complete dataset)*

```
Sample counts (n_k): {'mbpp': 245, 'humaneval': 123, 'ds1000': 750}
Weights (alpha_k): {'mbpp': 0.21914132379248658, 'humaneval': 0.11001788908765653, 'ds1000': 0.6708407871198568}
```

***Figure 6.2.a**: Output of fine-tuned adapters after Federated average (Training dataset)*

```
Final alpha_k: {'mbpp': 0.17893353222221403, 'humaneval': 0.04168543557403319, 'ds1000': 0.7793810322037528}
```

***Figure 6.2.b**: Output of fine-tuned adapters after Federated Error average (Training dataset)*

Module 4: Verification Module

The model is evaluated across all three datasets using four adapter variants: FedAvg Complete, FedAvg Train, FedErrorAvg Complete, and FedErrorAvg Train. The "Complete" variants are trained and tested on the full dataset, while the "Train" variants are trained on the 75% training partition and evaluated on the held-out 25% test partition.

A key aspect of this evaluation framework is that the base model weights remain entirely frozen throughout the process. The federated adapter weights — aggregated LoRA matrices from across all clients — are applied as an additive overlay on top of the frozen base. This means the base model's pre-trained knowledge is preserved, and the adapter contributes only the task-specific delta learned during fine-tuning. Any observed improvement in accuracy is therefore attributable entirely to the federated LoRA adapters, cleanly isolating the contribution of the federated fine-tuning process.

```python
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import PeftModel
import torch
from tqdm import tqdm

base_id = "Qwen/Qwen2.5-Coder-3B-Instruct"
try:
    tokenizer = AutoTokenizer.from_pretrained(ADAPTER_PATH, trust_remote_code=True)
except Exception:
    tokenizer = AutoTokenizer.from_pretrained(base_id, trust_remote_code=True)
base_model = AutoModelForCausalLM.from_pretrained(base_id, device_map="auto", trust_remote_code=True)
model = PeftModel.from_pretrained(base_model, ADAPTER_PATH)
model.eval()
print("Model and tokenizer loaded.")
```

***Figure** 7: Base Model import and Tokeniser Loading*

Module 5: Fed-Average Algorithm

The module implements the proposed Fed-Error Average algorithm, which extends standard Federated Averaging by incorporating hallucination error frequency information alongside dataset size.

Algorithm 1 Fed-Error Average: Frequency-Weighted Federated Aggregation

Require: Client LoRA adapters $\{w_k^t\}_{k=1}^K$, error counts $\{\text{count}_{k,e}\}$
Ensure: Global adapter w^{t+1}

Step 1: Compute error frequency for each error type e
    $F_e \leftarrow \sum_{k=1}^{K} \text{count}_{k,e}$
Step 2: Compute error-type weight
    $\text{weight}_e \leftarrow \frac{F_e}{\sum_{e' \in \mathcal{E}} F_{e'}}$
Step 3: Compute client error score E_k
    $E_k \leftarrow \sum_{e \in \mathcal{E}} \text{count}_{k,e} \cdot \text{weight}_e$
Step 4: Normalize client weights
    $\alpha_k \leftarrow \frac{E_k}{\sum_{j=1}^{K} E_j}$
Step 5: Aggregate client adapters
    $w^{t+1} \leftarrow \sum_{k=1}^{K} \alpha_k \, w_k^t$
return w^{t+1}

The advantage of this formulation over standard FedAvg is that clients whose local datasets reflect more complex or more diverse hallucination patterns for instance, a client that observed many distinct error types versus one that saw mostly a single error type contribute proportionally more to the global update. This is particularly important in heterogeneous federated settings, where different clients encounter qualitatively different failure modes in code generation.

Formulae Used

1. Federated average

This is the conventional practice in federated learning where the size of the dataset is used for the calculation of the new weight.

$$w^{t+1} = \sum_{k=1}^{K} \frac{n_k}{N} w_k^t$$

K - client

n_k = number of training samples on client k

N – total number of samples used across all clients

2. Augmented Fed-average(Frequency-weighted/Federated-error average)

This work introduces an advanced variant of the Federated Averaging algorithm, designed to assign higher weights based on the frequency and occurrence of specific error types. By incorporating error-aware weighting into the aggregation process, the proposed method prioritizes consistently observed failure patterns, leading to a more informed global update. As a result, this approach outperforms traditional Federated Averaging in code generation tasks, particularly in reducing hallucinations and improving overall output reliability.

Error frequency;

$$F_e = \sum_{k=1}^{K} count_{k,e}$$

F_e = total occurrences of error type ‘e’ across all clients

$count_{k,e}$ = number of errors of type ‘e’ in client k

Weight Definition:

$$weight_e = \frac{F_e}{\sum_{e' \in errors} F_{e'}}$$

This is calculated for weight for each type of error based on their occurrence.

Error score:

For each client, the total count of each type of error along with its computed weight is accounted for Normalization,

$$E_k = \sum_{e \in errors} count_{k,e} \cdot weight_e$$

$$\alpha_k = \frac{E_k}{\sum_{j=1}^{K} E_j}$$

Augmented FedAverage,

$$w^{t+1} = \sum_{k=1}^{K} \alpha_k w_k^t$$

w_k^t - local model weight on round t on client k
α_k - normalized weight assigned to client k
w^{t+1} - Global model weights after aggregation at round $t + 1$

Results and Analysis

Pass@1 Performance Comparison: Baseline vs. Federated Averaging

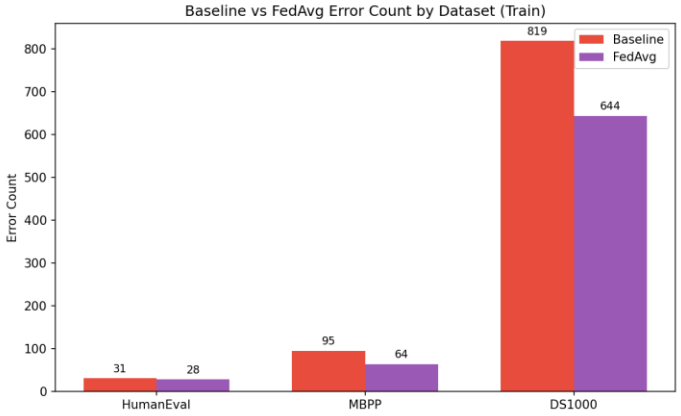

Figure 8: *Baseline vs FedAverage Error count by Dataset*

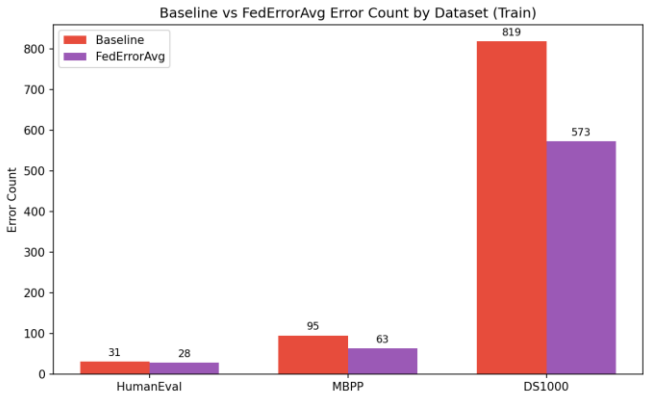

Figure 9: *Baseline vs Fed-Error Average Error count by Dataset*

Figures 8 and 9 illustrate the number of hallucinated problems across each dataset for the frozen base model, as well as for adapters fine-tuned using Federated Averaging and the proposed Fed-Error Averaging method. While both approaches demonstrate a reduction in hallucinations compared to the base model, Fed-Error Averaging achieves a more substantial decrease, indicating its superior effectiveness in code generation.

Pass rate comparison

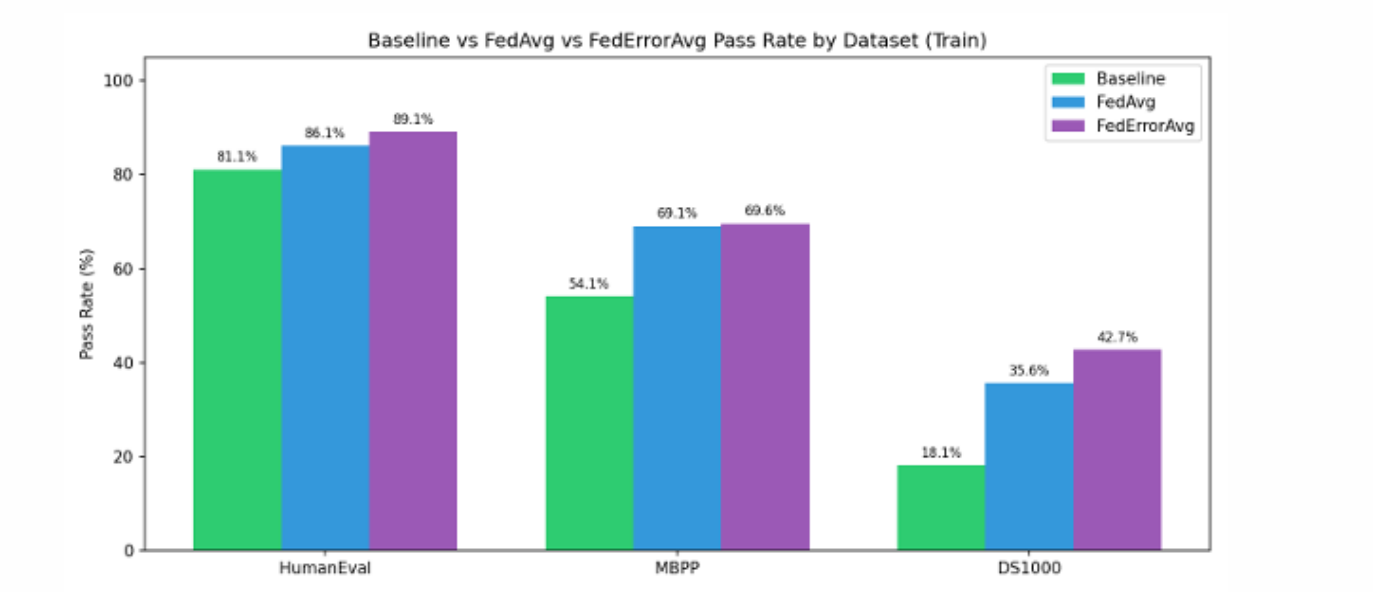

Figure 10: *Pass rate comparison across three datasets, evaluating baseline performance against Federated Averaging and Error-Aware Federated Averaging methods.*

Error Type	baseline_count	fedavg_count	federroravg_count	reduction
AssertionError	*379*	*364*	*365*	*14*
AttributeError	*58*	*43*	*40*	*18*
AxisError	*4*	*1*	*0*	*4*
FileNotFoundError	*3*	*0*	*0*	*3*
IndentationError	*69*	*48*	*60*	*9*
IndexError	*6*	*9*	*9*	*0*
IndexingError	*2*	*0*	*0*	*2*
InvalidIndexError	*1*	*0*	*0*	*1*
KeyError	*19*	*54*	*18*	*1*
LinAlgError	*1*	*0*	*0*	*1*
ModuleNotFoundError	*2*	*1*	*2*	*0*
NameError	*201*	*56*	*40*	*161*
NotFittedError	*6*	*5*	*2*	*4*
NotImplementedError	*2*	*1*	*1*	*1*
OverflowError	*0*	*1*	*0*	*0*
QhullError	*0*	*1*	*1*	*0*
RecursionError	*0*	*1*	*0*	*0*
RuntimeError	*11*	*7*	*7*	*4*
SyntaxError	*24*	*4*	*4*	*20*
TimeoutError	*14*	*0*	*0*	*14*
TypeError	*80*	*85*	*84*	*0*
UFuncTypeError	*0*	*4*	*4*	*0*
UnboundLocalError	*1*	*0*	*0*	*1*
UnidentifiedImageError	*1*	*0*	*0*	*1*
ValueError	*61*	*62*	*55*	*6*

return_outside_function	*0*	*16*	*6*	*0*

Table 5*: Total Error Reduction after application of FedAverage vs Fed-Error Average*

Fine-tuning again shows a strong overall improvement, with most major error types decreasing significantly, especially under FedErrorAvg. The most impactful reduction is in NameError (201 → 40), a drop of 161 errors, indicating that the model has learned variable scoping and definitions much better. Similarly, SyntaxError (24 → 4) and TimeoutError (14 → 0) are almost eliminated, showing clear gains in code structure and execution reliability. Errors like AttributeError, NotFittedError, AxisError, and FileNotFoundError also reduce consistently, suggesting improved API usage and better alignment with library-specific behaviors. Overall, these reductions highlight that fine-tuning greatly improves basic correctness, structure, and executability of generated code.

CodeBLEU Similarity

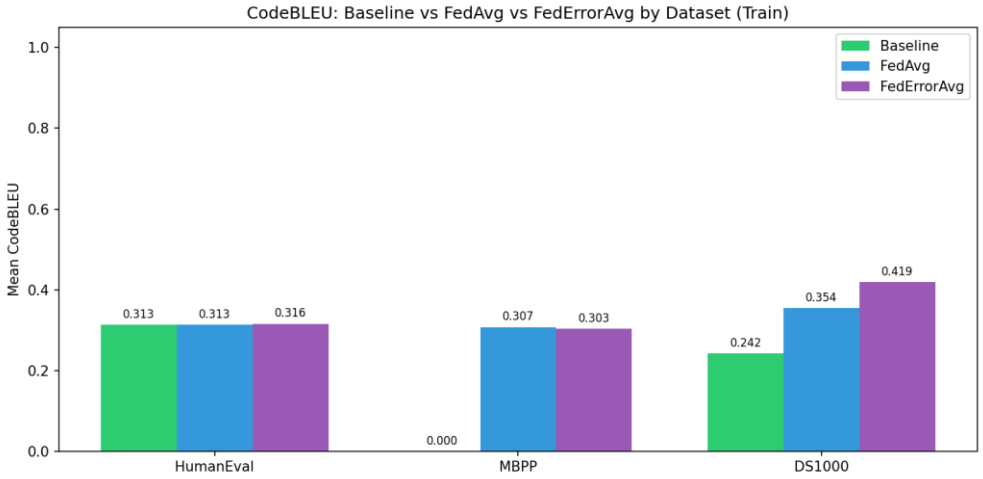

Figure 11: *Baseline vs Fed-Error Average Error count by Dataset*

CodeBLEU is a metric for evaluating the quality of AI-generated code against reference code. It's an extension of the classic BLEU score but specifically designed for code.

From our analysis with both FedAvg and FedErrorAvg outperform the Baseline on all three datasets. On HumanEval the scores are very close (0.292 → 0.299 → 0.297), showing fine-tuning gives a small but consistent bump. On MBPP the baseline is a striking 0.000, meaning the pre-fine-tuned model generated code with zero meaningful similarity to reference solutions fine-tuning completely rescued this benchmark, pushing it to 0.284 and 0.295. On DS1000 the gains are the largest, with FedErrorAvg hitting 0.342 against a baseline of 0.241, roughly a 42% relative improvement.

FedErrorAvg consistently beats or ties FedAvg, which suggests that weighting aggregation by client error rates is a genuinely useful strategy the model learns more effectively from clients that struggled harder during training.This proves that fine-tuning works.

REFERENCES

[1] B. Yang, J. Dang, H. Liu and Z. Jin, "Advancing LLM-Generated Code Reliability: A Hybrid Approach for Hallucination Detection" in IEEE Transactions on Software Engineering, vol. , no. 01, pp. 1-17, PrePrints 5555, doi: 10.1109/TSE.2025.3640641.

[2] S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty and S. K. Lahiri, "LLM-Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation," in *IEEE Transactions on Software Engineering*, vol. 50, no. 9, pp. 2254-2268, Sept. 2024, doi: 10.1109/TSE.2024.3428972.

[3] J. Chen, Z. Cai, W. Chen, W. Wang, Z. Zheng and P. S. Yu, "A Federated Adaptive Large Language Model Fine-Tuning Framework for Software Development," in *IEEE Transactions on Services Computing*, doi: 10.1109/TSC.2025.3623626

[4] M. Nashaat and J. Miller, "Towards Efficient Fine-Tuning of Language Models With Organizational Data for Automated Software Review," in *IEEE Transactions on Software Engineering*, vol. 50, no. 9, pp. 2240-2253, Sept. 2024, doi: 10.1109/TSE.2024.3428324

[5] R. Khojah, F. G. de Oliveira Neto, M. Mohamad and P. Leitner, "The Impact of Prompt Programming on Function-Level Code Generation," in *IEEE Transactions on Software Engineering*, vol. 51, no. 8, pp. 2381-2395, Aug. 2025, doi: 10.1109/TSE.2025.3587794

[6] Q. Wang, C. Li, Y. Liu, Q. Zhu, J. Song and T. Shen, "An Adaptive Framework Embedded With LLM for Knowledge Graph Construction," in *IEEE Transactions on Multimedia*, vol. 27, pp. 2912-2923, 2025, doi: 10.1109/TMM.2025.3557717

[7] X. Yu, L. Liu, X. Hu, J. W. Keung, J. Liu and X. Xia, "Fight Fire With Fire: How Much Can We Trust ChatGPT on Source Code-Related Tasks?," in *IEEE Transactions on Software Engineering*, vol. 50, no. 12, pp. 3435-3453, Dec. 2024, doi: 10.1109/TSE.2024.3492204

[8] X. Yi, C. Hu, B. Cai, H. Huang, Y. Chen and K. Wang, "FedALoRA: Adaptive Local LoRA Aggregation for Personalized Federated Learning in LLM," in *IEEE Internet of Things Journal*, vol. 12, no. 24, pp. 51854-51865, 15 Dec.15, 2025, doi: 10.1109/JIOT.2025.3582427

[9] Z. Liu, Y. Tang, X. Luo, Y. Zhou and L. F. Zhang, "No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT," in *IEEE Transactions on Software Engineering*, vol. 50, no. 6, pp. 1548-1584, June 2024, doi: 10.1109/TSE.2024.3392499

[10] S. Ouyang, J. M. Zhang, Z. Sun and A. M. Penuela, "Knowledge-Enhanced Program Repair for Data Science Code," in 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), Ottawa, ON, Canada, 2025, pp. 898-910, doi: 10.1109/ICSE55347.2025.00246.

[11] J. Liu, Y. Zhang, D. Wang, Y. Li and W. Dong, "THINK: Tackling API Hallucinations in LLMs via Injecting Knowledge," *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, Montreal, QC, Canada, 2025, pp. 229-240, doi: 10.1109/SANER64311.2025.00029.

[12] H. Zhang, Q. Yan, W. Min, C. Zhang, L. Kuang and Y. Xia, "EvoAPR: Enhancing Large Language Models for Automatic Program Repair with Genetic Algorithm and Dynamic LoRA," *2025 IEEE International Conference on Web Services (ICWS)*, Helsinki, Finland, 2025, pp. 946-956, doi: 10.1109/ICWS67624.2025.00123.

[13] X. Yi, C. Hu, B. Cai, H. Huang, Y. Chen and K. Wang, "FedALoRA: Adaptive Local LoRA Aggregation for Personalized Federated Learning in LLM," in *IEEE Internet of Things Journal*, vol. 12, no. 24, pp. 51854-51865, 15 Dec.15, 2025, doi: 10.1109/JIOT.2025.3582427